Arquitectura de computadoras

RARO SUPREME



CUCEI

Clave: IL365 NRC: 200204 Seccion: D03 Calendario: 2026B

Ariel Rojas Garcia

221930873

1. INTRODUCCIÓN

1.1 La instrucción como palabra de control

En un procesador, una instrucción es una palabra de bits dividida en campos. Cada campo controla una parte distinta del hardware: uno indica qué dirección de memoria leer, otro si se debe escribir y otro qué operación realiza la ALU. En MIPS, por ejemplo, las instrucciones de 32 bits se dividen en campos como opcode, rs, rt, rd y funct.

En esta actividad se usa un formato didáctico propio de 23 bits, que no corresponde a una instrucción MIPS real. Su propósito es mostrar cómo los bits de una instrucción viajan por el hardware: al cambiar la instrucción seleccionada, cambian las direcciones leídas, la operación de la ALU y el resultado.

1.2 Jerarquía de módulos e instanciación

En Verilog un diseño grande se construye uniendo módulos pequeños. Instanciar un módulo significa colocar una copia de él dentro de otro y conectar sus puertos a cables del módulo contenedor. Con la conexión por nombre, cada puerto se indica explícitamente con la forma **.puerto(cable)**, de modo que el orden de los puertos no importa y es más difícil conectar algo en el lugar equivocado.

Cuando dos instancias comparten el mismo cable, la salida de una se convierte en la entrada de la otra. Así se forma un camino de datos (datapath): los datos salen de las memorias, pasan por la ALU y producen un resultado.

1.3 Memorias ROM y RAM en Verilog

Una memoria se describe como un arreglo de registros, **reg [ancho] nombre [profundidad]**. La ROM solo se lee: su salida se asigna de forma continua (**assign**), por lo que la lectura es combinacional. La RAM puede leerse o escribirse según la señal de habilitación de escritura (WE). El contenido inicial de ambas se carga desde archivos de texto en binario con la tarea de sistema **$readmemb**.

1.4 Complemento a dos y comparación con signo

El complemento a dos es la representación usada para los enteros con signo: si el bit más significativo es 1, el número es negativo. Por ejemplo, 32 bits en 1 (**0xFFFFFFFF**) representan −1, y **0x80000000** representa −2,147,483,648, el número más negativo de 32 bits.

En Verilog, los tipos **wire** y **reg** son sin signo por defecto. Por eso una comparación como **A < B** interpreta **0xFFFFFFFF** como 4,294,967,295 y no como −1. Para comparar con signo se usa la

función **$signed()**, que indica al simulador que trate los bits como un número en complemento a dos.

2. OBJETIVO

Objetivo general

Construir en Verilog un sistema que ejecute instrucciones propias de 23 bits conectando, mediante instancias, los módulos ROM, RAM y MIPSALU desarrollados en clase, y verificar su funcionamiento por simulación seleccionando cada una de las 16 instrucciones de una ROM de instrucciones.

Objetivos particulares

- Separar los cinco campos de la instrucción (DROM, DRAM, WE, DataIn y OP) en el módulo RARO_SUPREME.
- Instanciar ROM, RAM y MIPSALU con conexiones por nombre, enviando la salida de la ROM a la entrada A y la salida de la RAM a la entrada B de la ALU.
- Describir ROM_INSTRUCCIONES como una memoria de 16 palabras de 23 bits con lectura combinacional, cargada desde un archivo con **$readmemb**.
- Construir SISTEMA_RARO, donde un selector de 4 bits elige la instrucción a ejecutar, sin reloj ni contador.
- Decodificar a mano las instrucciones antes de simular y comparar las predicciones con la simulación.
- Verificar suma, resta, SLT con signo, resultado cero, resultado negativo de 32 bits y la escritura y lectura posterior en RAM.

3. DESARROLLO

3.1 Formato de la instrucción

La instrucción tiene 23 bits divididos en cinco campos (6 + 6 + 1 + 6 + 4). Las direcciones son de 6 bits porque cada memoria de datos tiene 64 palabras de 32 bits.

Campo	Bits	Ancho	Uso
DROM	22:17	6	Dirección de la ROM de datos; su valor es el operando A

DRAM	16:11	6	Dirección de la RAM de datos; su valor leído es el operando B
WE	10	1	1 escribe en la RAM; 0 lee y usa la ALU
DataIn	9:4	6	Valor de 0 a 63 que se escribe en la RAM
OP	3:0	4	Operación de la ALU: 2 suma, 6 resta, 7 SLT con signo

Tabla 1. Campos de la instrucción de 23 bits.

Al escribir, DataIn se extiende con 26 ceros a la izquierda (**{26'b0, DataIn}**) para formar la palabra de 32 bits que recibe la RAM.

3.2 Contenido inicial de las memorias

La ROM de datos se carga desde el archivo **memrom** y la RAM desde **memram**; ambos contienen 64 palabras de 32 bits en binario. Las direcciones que no aparecen en la tabla valen 0.

Dirección	**ROM (memrom)**	**RAM (memram)**
0	5	3
1	10	10
2	0	0
3	0xFFFFFFFF (−1)	7

4	0x80000000 (−2,147,483,648)	0xFFFFFFFF (−1)
5	40	120
63	63	1

Tabla 2. Valores iniciales de las memorias de datos (decimal con signo).

La ROM de instrucciones se carga desde **mem_instrucciones**, que contiene las 16 instrucciones de 23 bits del programa.

3.3 Módulos desarrollados

ROM_INSTRUCCIONES (ROM_INSTRUCCIONES.v). Es una memoria de 16 palabras de 23 bits. La dirección de 4 bits permite exactamente 16 posiciones (2^4). La lectura es combinacional mediante **assign**: en cuanto cambia la dirección, cambia la instrucción de salida.

RARO_SUPREME (RARO_SUPREME.v). Recibe la instrucción completa y la corta en sus cinco campos con selección de bits (**instruccion[22:17]**, etc.). Declara dos cables de 32 bits, **c1** para el dato de la ROM y **c2** para el dato de la RAM, que funcionan como puentes hacia la ALU. Las tres instancias se conectan por nombre:

- **bombon** (ROM): recibe DROM y entrega **c1**.
- **burbuja** (RAM): recibe DRAM, WE y **DataIn32**, y entrega **c2**.
- **bellota** (MIPSALU): recibe OP, A = **c1** y B = **c2**; su resultado va a la salida DD y su bandera Zero a ZF.

SISTEMA_RARO (RARO_SUPREME.v). Une la ROM de instrucciones con RARO_SUPREME. El selector de 4 bits es la dirección de la ROM de instrucciones. La salida **instruccion** sirve al mismo tiempo de puente entre las dos instancias y de salida observable para el testbench. No hay reloj, contador ni avance automático.

Módulos de clase (ROM.v, RAM.v y MIPSALU.v). La ROM y la RAM son memorias de 64 palabras de 32 bits cargadas con **$readmemb** desde **memrom** y **memram**. La RAM escribe **DataIn** en **mem[DirL]** cuando **we = 1** y, cuando **we = 0**, entrega **mem[DirL]** en **DataOut**. En MIPSALU se modificó la operación SLT para que compare con signo (sección 3.7).

Testbench (RARO_SUPREME_TB.v). Recorre el selector de 0 a 15, espera 10 ns en cada valor para que el circuito se estabilice e imprime la instrucción en binario, DD en hexadecimal y ZF.

3.4 Diagrama de instancias

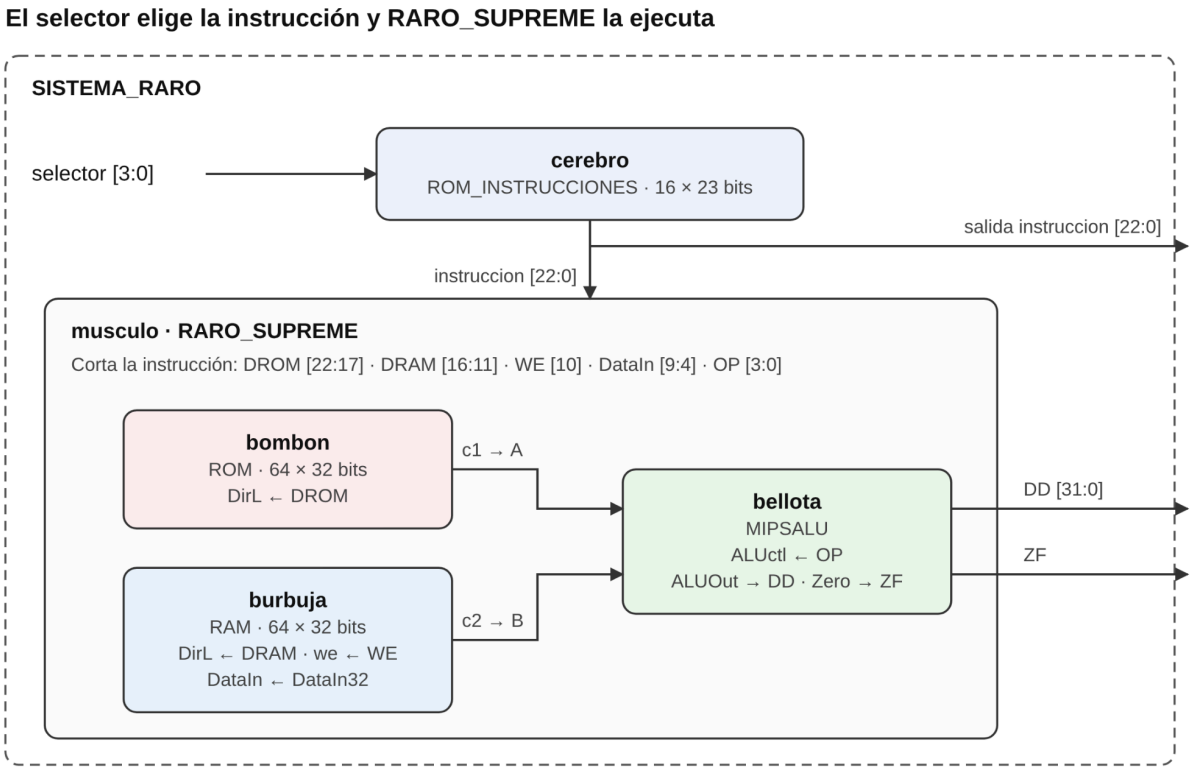


Figura 1. Diagrama de instancias de SISTEMA_RARO.

El selector entra a **cerebro**, cuya instrucción llega a **musculo** y también sale como observable. Dentro de **musculo**, **bombon** entrega A por **c1**, **burbuja** entrega B por **c2** y **bellota** produce DD y ZF.

3.5 Predicciones antes de simular

Antes de simular se decodificaron a mano las instrucciones pedidas. Para cada una se cortó la palabra en sus cinco campos, se buscó el valor de cada dirección en **memrom** y **memram** y se calculó el efecto esperado.

Selector	Campos (DROM, DRAM, WE, DataIn, OP)	Operandos	Efecto que predigo
0	0, 0, 0, 0, 2	A = ROM[0] = 5, B = RAM[0] = 3	Suma: DD = 8, ZF = 0
2	3, 0, 0, 0, 7	A = ROM[3] = −1, B = RAM[0] = 3	SLT: −1 < 3, DD = 1, ZF = 0
6	0, 2, 1, 63, 2	A = 5, B = RAM[2] = 0	Escribe 63 en RAM[2]; DD y ZF no se califican
7	1, 2, 0, 0, 2	A = ROM[1] = 10, B = RAM[2] = 63 (escrito por la 6)	Suma: DD = 73, ZF = 0
10	0, 0, 1, 0, 2	A = 5, B = RAM[0] = 3	Escribe 0 en RAM[0]; DD y ZF no se califican
11	0, 0, 0, 0, 2	A = 5, B = RAM[0] = 0 (escrito por la 10)	Suma: DD = 5, ZF = 0

Tabla 3. Decodificación manual.

3.6 Resultados de la simulación

Las 16 instrucciones se ejecutaron en ModelSim con el testbench RARO_SUPREME_TB. Todas las predicciones de la Tabla 3 coincidieron con la simulación final.

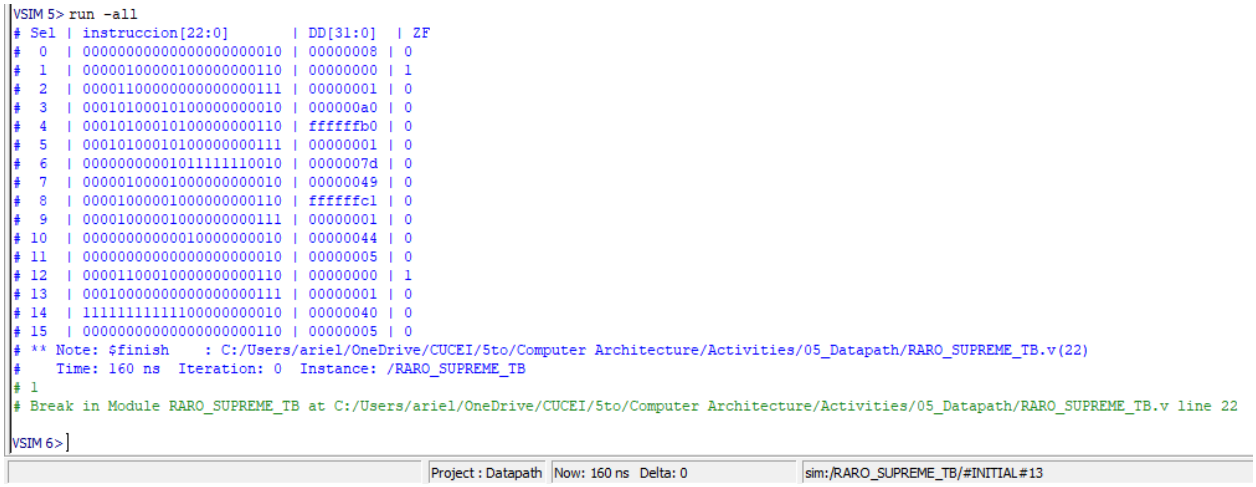


Figura 2. Registro de la simulación en el Transcript de ModelSim.

Sel	Campos (DROM, DRAM, WE, DataIn, OP)	A	B	Acción	DD (hex / decimal)	ZF
0	0, 0, 0, 0, 2	5	3	Suma	00000008 / 8	0
1	1, 1, 0, 0, 6	10	10	Resta	00000000 / 0	1
2	3, 0, 0, 0, 7	−1	3	SLT	00000001 / 1	0
3	5, 5, 0, 0, 2	40	120	Suma	000000a0 / 160	0
4	5, 5, 0, 0, 6	40	120	Resta	fffffb0 / −80	0
5	5, 5, 0, 0, 7	40	120	SLT	00000001 / 1	0

6	0, 2, 1, 63, 2	—	—	Escribe 63 en RAM[2]	no se califica	—
7	1, 2, 0, 0, 2	10	63	Suma	00000049 / 73	0
8	2, 2, 0, 0, 6	0	63	Resta	fffffc1 / −63	0
9	2, 2, 0, 0, 7	0	63	SLT	00000001 / 1	0
10	0, 0, 1, 0, 2	—	—	Escribe 0 en RAM[0]	no se califica	—
11	0, 0, 0, 0, 2	5	0	Suma	00000005 / 5	0
12	3, 4, 0, 0, 6	−1	−1	Resta	00000000 / 0	1
13	4, 0, 0, 0, 7	−2,147,483,648	0	SLT	00000001 / 1	0
14	63, 63, 0, 0, 2	63	1	Suma	00000040 / 64	0
15	0, 0, 0, 0, 6	5	0	Resta	00000005 / 5	0

Tabla 4. Instrucciones decodificadas, operandos y resultados de las 16 pruebas.

3.7 Corrección de SLT con signo

En la primera simulación, los selectores 2 y 13 no coincidieron con lo esperado: ambos dieron DD = 0 y ZF = 1, cuando la predicción era DD = 1 y ZF = 0.

La causa es que la línea original **A < B ? 1:0** compara A y B sin signo, porque en Verilog los **wire** son sin signo por defecto. Así, **0xFFFFFFFF** se interpretaba como 4,294,967,295 y **0x80000000** como 2,147,483,648, ambos mayores que B. Ambas filas tienen OP = 7 y una dirección de ROM cuyo valor empieza con 1 en el bit más significativo.

La corrección consistió en usar **$signed()** en la comparación:

```
4'd7: ALUOut <= ($signed(A) < $signed(B)) ? 1:0;
```

Después de recompilar, los selectores 2 y 13 dieron DD = 1 y ZF = 0, y el resto de la tabla no cambió.

3.8 Instrucciones 6 y 7

Instrucción 6: 000000 000010 1 111111 0010

- Bit 12 en 1: DRAM = 2, la dirección de la RAM que se va a escribir.
- Bit 10 en 1: WE = 1, la RAM está en modo escritura.
- Bits 9 a 4 en 1: DataIn = 111111 = 63, el dato a escribir, que se extiende a 32 bits como **{26'b0, 6'b111111}**.
- Bit 1 en 1: OP = 2 (suma), aunque el resultado no se usa durante la escritura.

Efecto en la RAM: RAM[2] cambia de 0 a 63.

Instrucción 7: 000001 000010 0 000000 0010

- Bit 17 en 1: DROM = 1, así que A = ROM[1] = 10.
- Bit 12 en 1: DRAM = 2, la misma dirección que escribió la instrucción 6.
- Bit 10 en 0: WE = 0, la RAM lee y la ALU calcula.
- Bit 1 en 1: OP = 2, suma.

Cómo aparece el nuevo dato en el resultado: la RAM entrega RAM[2] = 63 como operando B, y la ALU suma 10 + 63 = 73, que aparece en DD como **00000049**, con ZF = 0.

3.9 Preguntas de la actividad

¿Qué cambia entre una instrucción de lectura y una de escritura? Lo que cambia es el bit WE. Con WE = 0, la RAM entrega el dato de la dirección DRAM, la ALU calcula A op B y el resultado aparece en DD y ZF. Con WE = 1, la RAM guarda DataIn en la dirección DRAM y no actualiza su salida, por lo que DD y ZF no tienen significado. En una lectura el campo DataIn no se usa, y en una escritura el campo OP no tiene efecto útil.

¿Por qué la instrucción 7 produce 73 después de ejecutar la 6? Porque la instrucción 6 escribe 63 en RAM[2] y la instrucción 7 lee esa misma dirección. El operando B ya no es el 0 original de **memram**, sino el 63 que quedó guardado, y 10 + 63 = 73. Las dos instrucciones se comunican a través de la memoria.

¿Qué sucedería si eligieras la 7 antes de la 6? RAM[2] todavía tendría su valor inicial de 0, por lo que la instrucción 7 calcularía 10 + 0 = 10 (**0000000a**), con ZF = 0. El resultado depende del orden en que se ejecutan las instrucciones, porque la RAM guarda el estado entre una y otra.

4. CONCLUSIÓN

Esta actividad me permitió ver cómo una instrucción no es más que un conjunto de números pegados, donde cada campo es la dirección o la orden para un bloque distinto del hardware. Al cortar la instrucción en DROM, DRAM, WE, DataIn y OP, y conectar cada campo a la ROM, la RAM o la ALU, entendí que los módulos que solo instancian otros módulos no calculan nada: su trabajo es conectar cables, y un mismo cable puede servir de puente entre la salida de una caja y la entrada de otra.

También aprendí la importancia de predecir el resultado antes de simular. Al decodificar a mano la instrucción 2, esperaba DD = 1 porque −1 es menor que 3, pero la simulación dio 0. Esa diferencia me llevó a revisar la ALU y descubrir que la comparación de SLT era sin signo, porque en Verilog los **wire** no tienen signo por defecto; con **$signed()** el resultado fue el correcto. Sin la predicción, ese error habría pasado como un resultado válido.

Por último, las instrucciones 6 y 7 mostraron que la RAM guarda estado entre instrucciones: una escribe un dato y la siguiente lo lee, así que el resultado depende del orden de ejecución. Aunque este sistema no usa reloj ni contador de programa, es un primer paso para entender cómo un procesador encadena instrucciones a través de la memoria.

5. REFERENCIAS

- Patterson, D. A., & Hennessy, J. L. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann.
- Harris, D. M., & Harris, S. L. *Digital Design and Computer Architecture*. Morgan Kaufmann.
- Mano, M. M., & Ciletti, M. D. *Diseño Digital*. Pearson Educación.
- Palnitkar, S. *Verilog HDL: A Guide to Digital Design and Synthesis*. Prentice Hall.
- IEEE. *IEEE Standard for Verilog Hardware Description Language* (IEEE Std 1364). IEEE Computer Society.
- Actividad RARO SUPREME — Arquitectura de Computadoras. Instancias e instrucciones de 23 bits (documento de la actividad).